

СОФИЙСКИ УНИВЕРСИТЕТ
„СВ. КЛИМЕНТ ОХРИДСКИ“



ФАКУЛТЕТ ПО МАТЕМАТИКА
И ИНФОРМАТИКА

ДЪРЖАВЕН ИЗПИТ

ЗА ПОЛУЧАВАНЕ НА ОКС „БАКАЛАВЪР ПО СОФТУЕРНО ИНЖЕНЕРСТВО“

ЧАСТ I (ПРАКТИЧЕСКИ ЗАДАЧИ)

Драги абсолвенти:

- Попълнете факултетния си номер в горния десен ъгъл на всички листове.
- Пишете само на предоставените листове, без да ги разкопчавате.
- Решението на една задача трябва да бъде на същия лист, на който е и нейното условие (т.е. може да пишете отпред и отзад на листа със задачата, но не и на лист на друга задача).
- Ако имате нужда от допълнителен лист, можете да поискате от квесторите.
- На един лист не може да има едновременно и чернова, и белова.
- Черновите трябва да се маркират, като най-отгоре на листа напишете „ЧЕРНОВА“.
- Ако решението на една задача не се побира на нейния лист, трябва да поискате нов бял лист от квесторите. Той трябва да се защити с телбод към листа със задачата.
- Всеки от допълнителните листове (белова или чернова) трябва да се надпише най-отгоре с вашия факултетен номер.
- Черновите също се предават и се защитават в края на работата.
- Времето за работа по изпита е 3 часа.

Изпитната комисия ви пожелава успешна работа!

Задача 1. Задачата да се реши на езика C++.

- а) Да се попълнят празните места в дадения долу фрагмент от програмен код така, че функцията `hasRepeatingDigits` да проверява рекурсивно дали в дадено цяло число има повтарящи се цифри.

```
bool hasRepeatingDigits(int number, bool digitCount[])
{
    if (number < 0){
        return _____;
    }

    if (number < 10) {
        return _____;
    }

    int digit = number % 10;

    if (_____) {
        return true;
    }

    _____;

    return hasRepeatingDigits(_____, _____);
}

bool hasRepeatingDigits(int number)
{
    bool digitCount[10] = { false };
    return hasRepeatingDigits(number, digitCount);
}
```

- б) Да се реализира функцията `collect` със следния прототип:

```
int* collect(int** matrix, int rows, int cols, int& resultSize);
```

Функцията да връща като резултат едномерен масив, съдържащ онези елементи на целочислената матрица `matrix` с размерност `rows × cols`, в които има повтарящи се цифри. Едномерният масив трябва да бъде заделен динамично с `new`. Размерът му да бъде точно колкото са елементите с повтарящи се цифри и да се връща чрез параметъра `resultSize`. Може да се използва наготово функцията `hasRepeatingDigits`, както и да се реализират допълнителни помощни функции.

- в) Да се реализира програма, която показва работата на функцията, като прочита от стандартния вход размерите на матрицата и елементите ѝ, извиква функцията `collect` и извежда на стандартния изход елементите на масива, който тя връща.

Примерно решение

```
a) bool hasRepeatingDigits(int number, bool digitCount[])
{
    if (number < 0){
        return hasRepeatingDigits(-number, digitCount);
    }

    if (number < 10) {
        return digitCount[number];
    }

    int digit = number % 10;

    if (digitCount[digit]) {
        return true;
    }

    digitCount[digit] = true;

    return hasRepeatingDigits(number / 10, digitCount);
}
bool hasRepeatingDigits(int number)
{
    bool digitCount[10] = { false };
    return hasRepeatingDigits(number, digitCount);
}

б) int* collect(int** matrix, int rows, int cols, int& resultSize)
{
    if (!matrix || rows <= 0 || cols <= 0)
        return nullptr;

    resultSize = 0;
    for (int r = 0; r < rows; ++r){
        for (int c = 0; c < cols; ++c){
            if (hasRepeatingDigits(matrix[r][c]))
                ++resultSize;
        }
    }

    int* result = new(std::nothrow) int[resultSize];
    if (!result) return nullptr;

    resultSize = 0;
    for (int r = 0; r < rows; ++r){
        for (int c = 0; c < cols; ++c){
            if (hasRepeatingDigits(matrix[r][c]))
                result[resultSize++] = matrix[r][c];
        }
    }
    return result;
}
```

в) Използваме функциите от горните подточки.

```
#include <iostream>

bool hasRepeatingDigits(int number);
int* collect(int** matrix, int rows, int cols, int& resultSize);

void clear(int** matrix, int rows)
{
    if (!matrix) return;
    for (int i = 0; i < rows; ++i)
        delete[] matrix[i];
    delete[] matrix;
}

int** createMatrix(int rows, int cols)
{
    int** matrix = nullptr;
    int r = 0;

    try{
        matrix = new int*[rows];
        for (r = 0; r < rows; ++r)
            matrix[r] = new int[cols];
    }catch (std::bad_alloc&){
        clear(matrix, r);
        return nullptr;
    }

    for (r = 0; r < rows; ++r){
        for (int c = 0; c < cols; ++c){
            std::cout << "matrix[" << r << "][" << c << "]=";
            std::cin >> matrix[r][c];
        }
    }
    return matrix;
}

int main()
{
    int rows = 0, cols = 0;
    do {
        std::cout << "Enter size:";
        std::cin >> rows >> cols;
    } while(rows <= 0 || cols <= 0);

    int** matrix = createMatrix(rows, cols);
    if(!matrix) {
        std::cout << "No memory!\n";
        return -1;
    }

    int size = 0;
    int* result = collect(matrix, rows, cols, size);
```

```
    if (result){
        for (int i = 0; i < size; ++i)
            std::cout << result[i] << ' ';
        std::cout << '\n';
        delete[] result;
    } else {
        std::cout << "No memory!\n";
    }

    clear(matrix, rows);
    return 0;
}
```

Критерии за оценяване

- а) За всяко вярно попълнено празно място се присъждат 0,5 точки.
 - б) За пълна и вярна реализация на функцията се присъждат 3 точки. Това включва изискване за коректно заделяне на динамична памет с точния размер.
 - в) За пълна и вярна реализация на главна функция се присъждат 4 точки. Това включва въвеждане на данни с подсигуряване на коректни стойности, заделяне на памет, изпълняване на функцията, извеждане на резултата и освобождаване на заделената памет за матрицата и резултатния масив.
-

Задача 2.

Задачата да се реши на езика C++.

А) Разгледайте дадения фрагмент.

```
1 class base {
2 public:
3     void print() {}
4 };
5
6 class derived : public base {
7     int* data;
8 public:
9     derived() { data = new int;}
10    ~derived() { delete data; }
11    void print() { std::cout << "123";}
12 };
13
14 void run() {
15     base* b = new derived();
16     b->print();
17     delete b;
18 }
```

A.1) Обяснете защо, въпреки че `run()` извиква `print()`, на изхода НЕ СЕ извежда "123".

A.2) Посочете къде в кода има изтичане на памет (memory leak) и обяснете защо (какво се случва).

A.3) Пренапишете кода на класа `base` така, че да НЕ изтича памет и `run()` да извежда на стандартния изход текста "123".

Б) Да се реализира следната йерархия:

Б.1) Абстрактен базов клас `item` с единствен член – публична константна чисто виртуална (pure virtual) функция `value`, която няма аргументи и връща стойност от тип `int`.

Б.2) Клас `num`, който наследява `item` и:

- пази в себе си число `K` от тип `int`;
- има конструктор, който получава число и инициализира с него `K`;
- реализира функцията `value()` да връща числото `K`.

Б.3) Клас `seq`, който наследява `item` и:

- има член-данна `private` динамично заделен с `new` масив от `N` числа от тип `int`;
- има конструктор, който получава число `N`, заделя масива и го запълва с псевдослучайни числа (използвайте `std::rand()`), като ако `N` не е положително число да се хвърля изключение;
- има всички функции от rule of 3 (копиращ конструктор, копиращо присвояване, деструктор) и управлява коректно паметта;
- реализира функцията `value()` да връща първия елемент на масива.

Критерии за оценяване

Сумата от точките се закръгля до цяло число.

А.1) 1 т. за напълно коректен отговор.

А.2) 1 т. за напълно коректен отговор.

А.3) 1 т. за напълно коректен отговор. Съгласно условието, признават се само решения, които коригират base. Не се признават решения, които коригират друг елемент (например функцията run()).

Б.1) 1 точка, ако класът е дефиниран коректно: функцията е публична, чисто виртуална (pure virtual), константна и с правилния тип.

Б.2) 1 точка, ако наследникът е дефиниран коректно и реализира нужната функционалност.

Б.3) По-долу е дадено колко точки носи всеки напълно коректен елемент на решението:

- 1 т. – конструктор, който коректно създава и инициализира масива;
- 1 т. – ако конструкторът от по-горната точка хвърля подходящо изключение ако се подаде некоректна стойност.
- 1 т. – копиращ конструктор;
- 1 т. – копиращо за присвояване;
- 0,5 т. – деструктор (вкл. освобождаване с delete[], а НЕ с delete);
- 0,5 т. – функцията value().

Примерно решение

Забележка: Решението е ориентировъчно. Отговорите може да се формулират по различен начин и да донесат пълни точки, стига да са верни.

А.1) print() НЕ Е виртуална функция. Свързването е статично и затова b->print() е обръщение към base::print(), а не на derived.

А.2) Аналогично, деструкторът на base НЕ Е виртуален. delete b води до обръщение към деструктора на base, а НЕ този на derived.

А.3)

```
class base {
public:
    virtual void print() {}
    virtual ~base() {}
};
```

Б.1)

```
class item {
public:
    virtual int value() const = 0;
};
```

Б.2)

```
class num : public item {
    int K;
public:
    num(int value) : K(value) {}
    virtual int value() const override {
        return K;
    }
};
```

Б.3)

```
class seq : public item {
    int* data;
    int size;
public:
    seq(int N) : size(N) {
        if(N <= 0)
            throw std::invalid_argument(
                "Incorrect size");

        data = new int[size];
        for(int i = 0; i < size; ++i)
            data[i] = std::rand();
    }

    seq(const seq& s) : size(s.size) {
        data = new int[size];
        for(int i = 0; i < size; ++i)
            data[i] = s.data[i];
    }

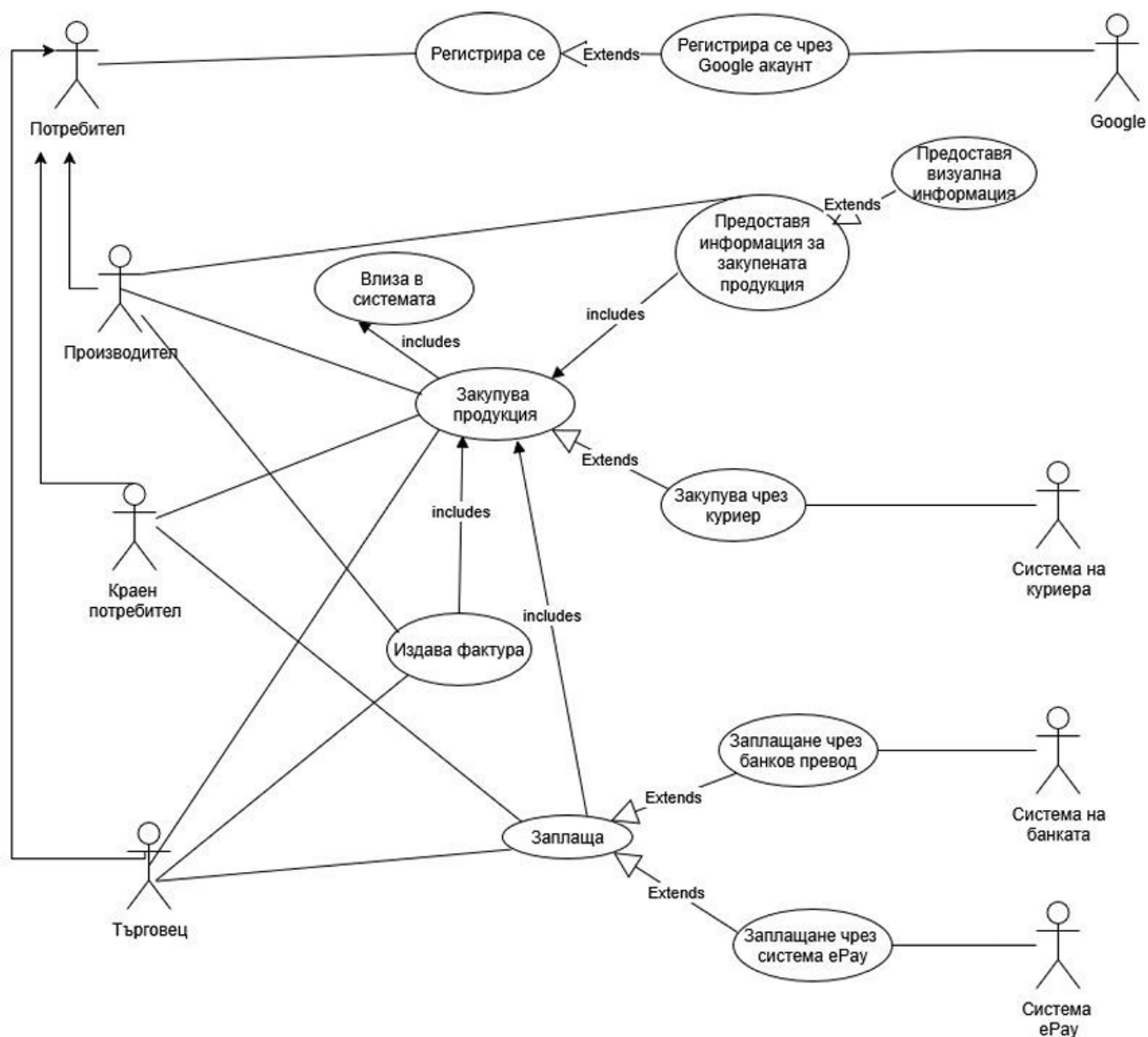
    seq& operator=(seq const& s) {
        if(this != &s) {
            seq temp(s);
            std::swap(data, s.data);
            std::swap(size, s.size);
        }
        return *this;
    }

    ~seq() { delete [] data; }

    virtual int value() const override {
        return data[0];
    }
};
```

Задача 3. Софтуерна система свързва производители на продукти, крайни потребители, търговци и куриерска фирма. Да се състави диаграма на потребителските случаи за следните изисквания на системата:

1. Всеки потребител на системата трябва да се регистрира в нея със съответна роля, което може да стане и чрез Google акаунт на потребителя.
 2. Системата позволява на регистрирани крайни потребители и търговци да закупят продукция на производителя, което може да се осъществи и чрез системата на куриерската фирма.
 3. Чрез системата производителят издава фактура за закупената стока на търговците.
 4. След закупуване на продукти производителят предоставя допълнителна текстова информация за закупената стока. Опционално информацията може да бъде допълнена с визуални материали.
 5. Системата предоставя възможност на потребителите на продукти да заплащат on-line закупена стока чрез банков превод или чрез системата ePay.
-

Примерно решение**Критерии за оценяване**

- А) За представени всички актьори – 4 точки. За всеки неотбелязан актьор точките се намаляват с 0,5;
- Б) За представени всички случаи на употреба – 4 точки. За неотбелязан случай на употреба точките се намаляват пропорционално;
- В) За записани вярно всички връзки – 2 точки. За неотбелязана/грешна връзка точките се намаляват пропорционално.
- За оформяне на оценката сумата от точките от А), Б) и В) се закръглява до цяло число.

Задача 4. Ценообразуването във фитнес клуб се извършва в съответствие със следните правила:

Ако клиентът има повече от 10 посещения на месец, може да се възползва от 5% отстъпка. Всеки клиент, който е член на клуба, получава 3% отстъпка. Ако е налична промоция, се добавя 3% отстъпка. Ако клиентът разполага с ваучер, се добавя 3% отстъпка само при условие, че не е налична текуща промоция.

Да се дефинира таблица за вземане на решение, въз основа на която могат да се генерират тестови сценарии. Таблицата трябва да показва условията, следствията от тях и правилата, от които се генерират тестовите сценарии.

Да се определи броя на тестовите сценарии в случай, че е възможно редуциране на правилата.

Въз основа на таблицата за вземане на решение да се определи резултатът от изпълнението на следните тестови сценарии:

ТС1: Член на клуба с ваучер за отстъпка с над 10 посещения и налична промоция.

ТС2: Клиент, който не е член на клуба и не притежава ваучер за отстъпка, с по-малко от 10 посещения и налична промоция.

ТС3: Клиент, който не е член на клуба, с над 10 посещения и налична промоция.

ТС4: Член на клуба с 10 посещения.

Примерно решение

Стъпка 1: Идентифицират се следните условия и следствия:

- C1: Клиент, член на клуба

C2: Повече от 10 посещения

C3: Налична промоция

C4: Клиент с ваучер за отстъпка

E1: Отстъпка 11%

E2: Отстъпка 8%

E3: Отстъпка 5%

E4: Отстъпка 3%

Стъпка 2: Условията и следствията се изброяват в таблица на решенията.

Стъпка 3: Броят на комбинациите е $2^3 = 8$.

Стъпка 4: Колоните на таблицата се попълват като се има предвид, че $RF1 = RF2 = 8/2 = 4, RF3 = 4/2 = 2, RF4 = 2/2 = 1$.

Стъпка 5: Определяне на следствията от всяко правило.

Стъпка 6: Определя се очаквания резултат от изпълнение на всяко следствие

Условия	Стойности	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
C ₁	T, F	T	T	T	T	T	T	T	T	F	F	F	F	F	F	F	F
C ₂	T, F	T	T	T	T	F	F	F	F	T	T	T	T	F	F	F	F
C ₃	T, F	T	T	F	F	T	T	F	F	T	T	F	F	T	T	F	F
C ₄	T, F	T	F	T	F	T	F	T	F	T	F	T	F	T	F	T	F
Действия																	
E ₁	11%	T	T	T													
E ₂	8%				T					T	T	T					
E ₃	5%					T	T	T					T				
E ₄	3%								T					T	T	T	
Checksum	16	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Стъпка 7: Редуцират се правилата

Условия	Стойности	1	2	3	4	5	6	7	8	9	10	11	12	13
C ₁	T, F, N/A	T	T	T	T	T	T	F	F	F	F	F	F	F
C ₂	T, F, N/A	T	T	T	F	F	F	T	T	T	F	F	F	F
C ₃	T, F, N/A	T	F	F	T	F	F	T	F	F	T	T	F	F
C ₄	T, F, N/A	N/A	T	F	N/A	T	F	N/A	T	F	T	F	T	F
Действия														
E ₁	11%	T	T											
E ₂	8%			T				T	T					
E ₃	5%				T	T				T				
E ₄	3%						T				T	T	T	
Checksum	16	2	1	1	2	1	1	2	1	1	1	1	1	1

След редуциране на правилата са необходими 13 тестови сценария.

Стъпка 8: Генерират се тестови сценарии, съответстващи на правилата в таблицата:

- Резултат ТС1: 11%

Резултат ТС2: 3%

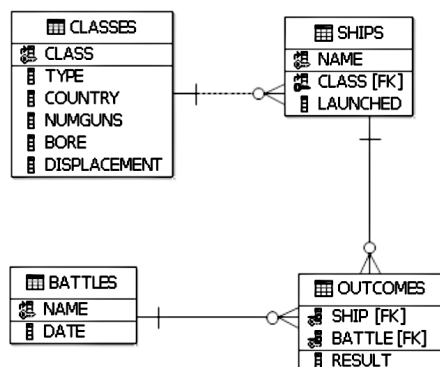
Резултат ТС3: 8%

Резултат ТС4: 3%

Критерии за оценяване

- Дефинирани условия и следствия: 1 т.
- Дефинирани стъпки за създаване на таблица за вземане на решения: 3 т.
- Създадена таблица за вземане на решения: 3 т.
- Редуциране на правила: 2 т.
- Определяне на резултати от примерните тестови сценарии: 1 т.

Задача 5. Дадена е базата от данни Ships, в която се съхранява информация за кораби (Ships) и тяхното участие в битки (Battles) по време на Втората световна война. Всеки кораб е построен по определен стереотип, определящ класа на кораба (Classes).



Таблицата Classes съдържа информация за класовете кораби:

- class — име на класа, първичен ключ;
- type — тип: 'bb' за бойни кораби и 'bc' за бойни крайцери;
- country — държавата, която строи такива кораби;
- numGuns — броят на основните оръдия;
- bore — калибърът им (диаметърът на отвора на оръдието в инчове);

• displacement — водоизместимост в тонове.
Таблицата Ships съдържа информация за корабите:

- name — име на кораб, първичен ключ;
- class — име на неговия клас;
- launched — годината, в която корабът е пуснат на вода.

Таблицата Battles съхранява информация за битките:

- name — име на битката, първичен ключ;
- date — дата на провеждане.

Таблицата Outcomes съдържа информация за резултата от участието на даден кораб в дадена битка, като колоните ship и battle заедно формират първичния ключ:

- ship — име на кораба;
- battle — име на битката;
- result — резултат от битката: 'sunk' за потънал, 'damaged' за повреден и 'ok' за победил.

1) Да се напише заявка, която извежда имената на битките и броя кораби, участвали във всяка битка.

2) Да се попълнят празните места в следната заявка така, че да се изведе името на класа и общата водоизместимост на всички кораби от този клас, които са пуснати на вода преди 1944 г.:

```
SELECT T1.class, _____ (T1.displacement) AS TotalDisplacement
FROM Classes AS T1
JOIN Ships AS T2 ON T1.class = T2.class
WHERE T2._____ < 1944
GROUP BY T1.class;
```

Примерно решение на подзадача 1

```
SELECT battle, COUNT(*) AS NumberOfShipsInBattle  
FROM Outcomes  
GROUP BY battle;
```

Примерно решение на подзадача 2

По ред на празните слотове:

- a) SUM
- б) launched

Критерии за оценяване

- 1) Общо 5 т., като за всяка сгрешена клауза броят на точките се намалява с 2 до достигане на 0 т.
- 2) Общо 5 т., по 2,5 т. на коректно посочен верен отговор.

Задача 6. Софтуерна система за видео стрийминг има следните изисквания:

- R1. Поддържа до 100 000 едновременни потребители без загуба на производителност.
- R2. Осигурява стрийминг на видеа с високо качество – както в HD, така и в 4K резолюция.
- R3. Позволява на потребителите да качват, управляват и актуализират своите видеа.
- R4. Предоставя функционалност за потребителски профили и сигурна автентикация.
- R5. Интегрира се с рекламни мрежи и други външни услуги, за да генерира приходи.
- R6. Гарантира ниска латентност при доставката на видео съдържание в световен мащаб.
- R7. Може лесно да се мащабира хоризонтално с нарастването на потребителската база.
- R8. Съхранява и управлява метаданни за видеосъдържание – заглавия, тагове, описания и др.
- R9. Позволява взаимодействие чрез коментари, харесвания и други потребителски действия.
- R10. Гарантира устойчивост при грешки, автоматично възстановяване и висока наличност на услугата.

Задание:

- 1. Да се предложи архитектурен стил (или комбинация от няколко), който би могъл да удовлетвори изискванията на системата, и да се обоснове защо този избор е подходящ спрямо нуждите на системата относно мащабируемост (scalability), наличност (availability), производителност (performance) и поддръжка (maintainability).
 - 2. Да се опишат възможните недостатъци, като например компромиси с други качествени изисквания и/или ограничения, които този избор би породил.
 - 3. Да се напишат два сценария за качество, за два качествени атрибута от следните три: наличност, производителност, изменчивост (modifiability). Решенията може да са представени както в свободен текст, така и с подточки (bullets.)
-

Примерно решение

Забележка: Решението е ориентировъчно. Отговорите може да се формулират по различен начин и да донесат пълни точки, стига да са верни.

Подусловие 1 (4 точки)

Архитектурен стил: Архитектура на Микроуслуги

Архитектурата на микроуслуги представлява архитектурен стил, при който приложението се разделя на малки, независимо работещи услуги. Всяка услуга изпълнява конкретна функционалност и комуникира с останалите чрез олекотени протоколи (напр. HTTP).

- **Мащабируемост:** Микроуслугите позволяват независимо мащабиране на отделни компоненти – например, видео обработка – според натоварването, без да е нужно това да се направи за цялата система.
- **Наличност:** При срыв на една услуга, останалите могат да продължат работа, без да се стига до срыв на цялата система, тъй като са независими една от друга.
- **Производителност:** Микроуслугите позволяват независимо мащабиране на отделни услуги според нуждите и разпределението на ресурси, което води до по-добро използване на ресурсите и потенциално по-бързо време за реакция.
- **Поддръжка:** Ясното разделение на отговорности между микроуслугите улеснява разработката, внедряването и поддръжката на системата от различни екипи, дори и да са разпределени в различни места.

Подусловие 2 (2 точки) Компромиси и ограничения:

- **Сложност:** Архитектурата на микроуслуги е по-сложна от монолитната – изисква управление на множество услуги, мрежова комуникация, оркестрация и мониторинг.
- **Комуникация:** При голям брой микроуслуги, комуникацията между тях може да стане източник на латентност, ако не е добре оптимизирана.
- **Тестване:** Интеграционното тестване на множество отделни компоненти може да е по-сложно и бавно.

- **Разработване:** Екипите трябва да имат опит с конкретни практики свързани с архитектурата на микроуслуги, като CI/CD и облачна инфраструктура.
- **Управление на състояние:** Необходимо е внимателно проектиране на състоянието и съгласуваността на данните, особено при евентуални грешки или частични сринове на услуги.

Подусловие 3 (4 точки) Сценарии за качество

1. Производителност. При нормално натоварване на системата, когато потребител избере видео за гледане, стриймингът трябва да започне за не повече от 2 секунди.

- **Въздействие:** избор на видео за гледане
- **Източник:** краен потребител
- **Контекст:** при нормално натоварване
- **Обект:** сървърен компонент за обработка и подаване на видео
- **Резултат:** системата започва стрийминг на видеото
- **Количествени параметри:** началното зареждане е ≤ 2 секунди

2. Наличност По време на пиков трафик, един от сървърите в системата се срива. Възстановяването на системата трябва да се извърши в рамките на 5 секунди, без загуба на повече от 2% от заявките.

- **Въздействие:** отказ на сървър
- **Източник:** системата (хардуерен отказ)
- **Контекст:** при високо натоварване / пиков трафик
- **Обект:** сървърната инфраструктура на системата
- **Резултат:** възстановяване на системата
- **Количествени параметри:** пренасочването се извършва за ≤ 5 секунди, без загуба на повече от 2% от заявките

Критерии за оценяване

- Подусловие 1 – 4 точки
- Подусловие 2 – 2 точки
- Подусловие 3 – 4 точки

Задача 7. Дадена е матрицата $A = \begin{pmatrix} -4 & 2 + 2\lambda & 2\lambda & 2\lambda \\ \lambda & 1 + \lambda & \lambda & \lambda \\ \lambda & 1 + \lambda & -2 & \lambda \\ -\lambda & -1 - \lambda & -\lambda & -2 - 2\lambda \end{pmatrix} \in M_4(\mathbb{R})$, където λ е реален параметър.

- а) Да се намери рангът на матрицата A в зависимост от стойностите на реалния параметър λ .
- б) Да се реши хомогенната система $Ax^t = 0$, където $x = (x_1, x_2, x_3, x_4) \in \mathbb{R}^4$ в зависимост от стойностите на реалния параметър λ (x^t е транспониран вектора x).
- в) Да се намери стойността на параметъра λ , ако векторът $b = (0, 2, -2, 1)$ е решение на системата $Ax^t = 0$.
- г) Да се намери фундаментална система решения за намерения параметър λ в подусловие в).

Примерно решение

- а) Да се намери рангът на матрицата A в зависимост от стойностите на реалния параметър λ .

Решение: Извършваме елементарни Гаусови преобразувания върху матрицата A , които я преобразуват последователно до необходимия еквивалентен вид, без да променят търсения ранг.

$$A \sim \begin{pmatrix} -2 & 1+\lambda & \lambda & \lambda \\ \lambda & 1+\lambda & \lambda & \lambda \\ \lambda & 1+\lambda & -2 & \lambda \\ -\lambda & -1-\lambda & -\lambda & -2-2\lambda \end{pmatrix} \sim \begin{pmatrix} -2 & \lambda+1 & \lambda & \lambda \\ \lambda+2 & 0 & 0 & 0 \\ 0 & 0 & \lambda+2 & 0 \\ 0 & 0 & 0 & \lambda+2 \end{pmatrix}$$

- 1) Разглеждаме случай $\lambda = -2$. Получаваме

$$A \sim \begin{pmatrix} -2 & -1 & -2 & -2 \\ 2 & 1 & 2 & 2 \end{pmatrix},$$

откъдето $r(A) = 1$.

- 2) Нека $\lambda \neq -2$, тогава A е еквивалентна на

$$A \sim \begin{pmatrix} -2 & \lambda+1 & \lambda & \lambda \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \sim \begin{pmatrix} 1 & 0 & 0 & 0 \\ -2 & \lambda+1 & \lambda & \lambda \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \sim \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \lambda+1 & \lambda & \lambda \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- 2.1) Разглеждаме случая $\lambda = -1$.

$$A \sim \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \sim \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Следователно $r(A) = 3$.

- 2.2) Нека $\lambda \neq -1$.

$$A \sim \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \lambda+1 & \lambda & \lambda \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

и $r(A) = 4$.

Окончателно, ако:

- 1) $\lambda = -2 \Rightarrow r(A) = 1$,
- 2) $\lambda = -1 \Rightarrow r(A) = 3$,
- 3) $\lambda \neq -1, -2 \Rightarrow r(A) = 4$.

- б) Да се реши хомогенната система $Ax^t = 0$, където $x = (x_1, x_2, x_3, x_4) \in \mathbb{R}^4$ в зависимост от стойностите на реалния параметър λ .

Решение: Преобразуванията, които извършихме в подточка а), бяха само по-редове, така може да използваме получените еквивалентни матрици при решаването на системата $Ax^t = 0$.

1) $\lambda = -2 \Rightarrow r(A) = 1,$

$$A \sim \begin{pmatrix} 2 & 1 & 2 & 2 \end{pmatrix},$$

Понеже преобразуванията са само по редове, то системата $Ax^t = 0$ е еквивалентна на

$$| 2x_1 + x_2 + 2x_3 + 2x_4 = 0.$$

Общият вид на решението на системата има вида:

$$(p, -2p - 2q - 2r, q, r).$$

2) $\lambda = -1 \Rightarrow r(A) = 3,$ В този случай системата е следната:

$$\left| \begin{array}{lcl} x_1 & & = 0 \\ & x_3 & = 0 \\ & & x_4 = 0 \end{array} \right. , \quad (1)$$

а решението е $(0, p, 0, 0).$

3) $\lambda \neq -1, -2 \Rightarrow r(A) = 4.$ В този случай системата е следната:

$$\left| \begin{array}{lcl} x_1 & & = 0 \\ & x_2 & = 0 \\ & & x_3 = 0 \\ & & & x_4 = 0 \end{array} \right. , \quad (2)$$

а решението е нулевото $(0, 0, 0, 0).$

в) Да се намери стойността на параметъра λ , ако векторът $b = (0, 2, -2, 1)$ е решение на системата $Ax^t = 0$.

Решение: Матрицата A е еквивалентна по редове на

$$A \sim \begin{pmatrix} -2 & \lambda + 1 & \lambda & \lambda \\ \lambda + 2 & 0 & 0 & 0 \\ 0 & 0 & \lambda + 2 & 0 \\ 0 & 0 & 0 & \lambda + 2 \end{pmatrix}$$

За да получим стойностите на параметъра λ , за които векторът $b = (0, 2, -2, 1)$ е решение на системата $Ax^t = 0$, трябва да се реши системата $Ab^t = 0$, т.е.:

$$\left| \begin{array}{lcl} 2(\lambda + 1) & -2\lambda & +\lambda = 0 \\ (\lambda + 2) \cdot 0 & & = 0 \\ & -2(\lambda + 2) & = 0 \\ & (\lambda + 2) & = 0 \end{array} \right. \Leftrightarrow \left| \begin{array}{l} \lambda + 2 = 0 \end{array} \right. \Leftrightarrow \lambda = -2.$$

г) Да се намери фундаментална система решения за намерения параметър λ в подусловие в).

Решение: При $\lambda = -2$, рангът на $r(A) = 1$. Матрицата A е еквивалентна на $(2, 1, 2, 2)$ по редове, т.е търсим фундаментална система решение (ФСР) на следната хомогенна система линейни уравнения:

$$| 2x_1 + x_2 + 2x_3 + 2x_4 = 0.$$

Общият вид на решението на системата, от б) е:

$$(p, -2p - 2q - 2r, q, r).$$

Една нейна фундаментална система от решения (ФСР) е

$$u_1 = (1, -2, 0, 0), \text{ при } p = 1, q = 0, r = 0$$

$$u_2 = (0, -2, 1, 0), \text{ при } p = 0, q = 1, r = 0$$

$$u_3 = (0, -2, 0, 1), \text{ при } p = 0, q = 0, r = 1.$$

.

Критерии за оценяване

При коректно и вярно решение на всяка от подточките се присъждат съответно:

а) 3 т.

б) 3 т.

в) 2 т.

г) 2 т.

Задача 8. Да се пресметне интегралът:

$$\int_0^1 x^5 \ln(x^4 + 1) dx$$

Примерно решение

Преобразуваме последователно:

$$\begin{aligned}
 \int_0^1 x^5 \ln(x^4 + 1) dx &= \\
 (1 \text{ т.}) \quad &= \frac{1}{6} \int_0^1 \ln(x^4 + 1) d(x^6) = \\
 (1 \text{ т.}) \quad &\quad \quad \quad (\text{полагаме } t = x^2) \quad \quad \quad = \frac{1}{6} \int_0^1 \ln(t^2 + 1) d(t^3) = \\
 (1 \text{ т.}) \quad &\quad \quad \quad (\text{интегрираме по части}) \quad \quad \quad = \frac{1}{6} [\ln(t^2 + 1)t^3]_0^1 - \frac{1}{6} \int_0^1 t^3 d[\ln(t^2 + 1)] = \\
 (1 \text{ т.} + 1 \text{ т.}) \quad &\quad \quad \quad = \frac{1}{6} [\ln(2) - 0] - \frac{1}{6} \int_0^1 t^3 \frac{2t}{t^2 + 1} dt = \\
 (1 \text{ т.}) \quad &\quad \quad \quad = \frac{1}{6} \ln(2) - \frac{1}{3} \int_0^1 \frac{t^4 + t^2 - t^2 - 1 + 1}{t^2 + 1} dt = \\
 (1 \text{ т.}) \quad &\quad \quad \quad = \frac{1}{6} \ln(2) - \frac{1}{3} \int_0^1 \left[t^2 - 1 + \frac{1}{t^2 + 1} \right] dt = \\
 (1 \text{ т.}) \quad &\quad \quad \quad = \frac{1}{6} \ln(2) - \frac{1}{3} \left[\frac{1}{3} t^3 - t + \arctg(t) \right]_0^1 = \\
 (1 \text{ т.}) \quad &\quad \quad \quad = \frac{1}{6} \ln(2) - \frac{1}{9} + \frac{1}{3} - \frac{1}{3} \cdot \frac{\pi}{4} = \\
 (1 \text{ т.}) \quad &\quad \quad \quad = \frac{1}{6} \ln(2) + \frac{2}{9} - \frac{\pi}{12}
 \end{aligned}$$

Критерии за оценяване

За всяка вярна стъпка от пресмятането се присъжда точка, съгласно посочените в примерното решение.

Чернова